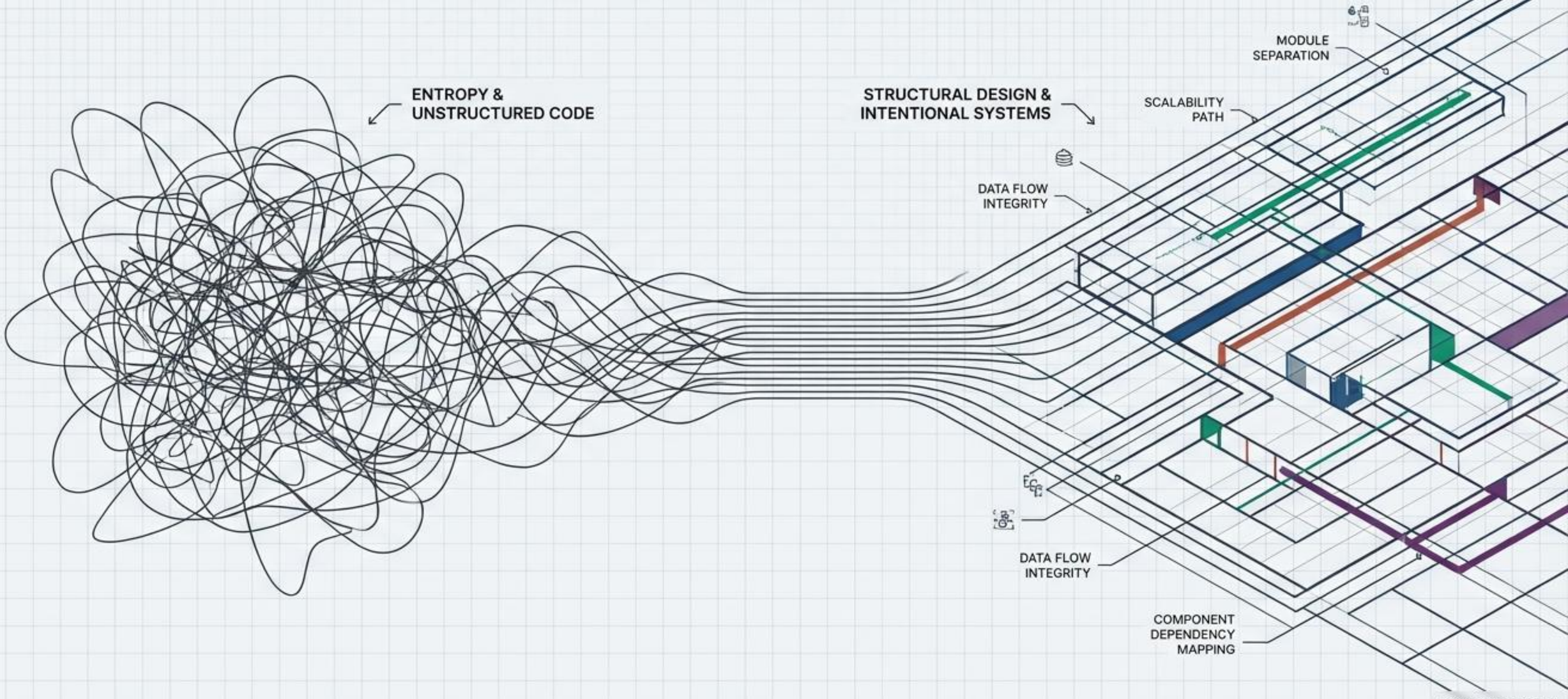
The Foundations of Software Architecture

Moving from “How” to “Why”



NotebookLM

The Architectural Sequence

Moving from abstract human decisions to concrete system building blocks.

The Role of the Architect
(The human agent)

Quality Attributes
(The rules of order)

Modularity Metrics
(Measuring the order)

Component-Based Thinking
(Implementing the order)

NotebookLM

The Architect

Definition

- Expensive Decisions
- Manage Complexity
- Technical Debt
- System Blueprint

Architect vs. Design

Macro (Structure / Topology)	Micro (Implementation / Logic)
High Impact (SQL vs. NuSQL)	Low Impact (Loops / Maps)

Expectations

- T-Shaped Professional (Breadth + Depth)
- Analyze & Track Trends
- Zero-Sum Negotiator

Stakeholder Management

Power \ Interest	
Manage Closely (High Power / High Interest)	Keep Satisfied (High Power / Low Interest)
Keep Informed (Low Power / High Interest)	Monitor (Low Power / Low Interest)

NotebookLM

The Architect

The Definition

- Making decisions that are expensive to change.
- Creating the blueprint of components and relationships.
- Managing system complexity and technical debt.

Core Expectations

- Make definitive architectural decisions.
- Ensure compliance with those decisions.
- Continually analyze the architecture.
- Keep current with shifting industry trends.

The T-Shaped Model

Horizontal: Broad technical breadth across multiple platforms.

Vertical: Deep specialization in specific technology stacks.

NotebookLM

Quality Attributes

ASRs

- Architecturally Significant Requirements
- Business Constraints
- Technical Constraints

ISO 25010

Operational
(Availability, Performance, Scalability)

Structural
(Maintainability, Extensibility)

Cross-Cutting
(Security, Usability)

6-Part Scenarios

Source → Stimulus → Artifact → Environment → Response → Measure

Prioritization

- Utility Trees
- ASR Matrix (Business Value vs. Impact)
- Tension Points (Trade-offs)

NotebookLM

Defining and Measuring Quality Attributes

System Quality

The '-ilities' (ISO 25010)

Core characteristics the system must support.

Reliability,
Scalability,
Security, Maintainability.

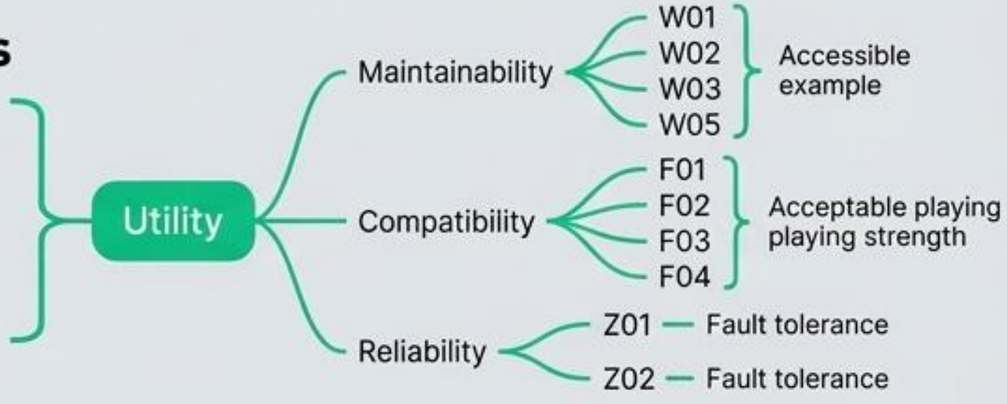
The Scenario Anatomy

Translating subjective needs into objective benchmarks.

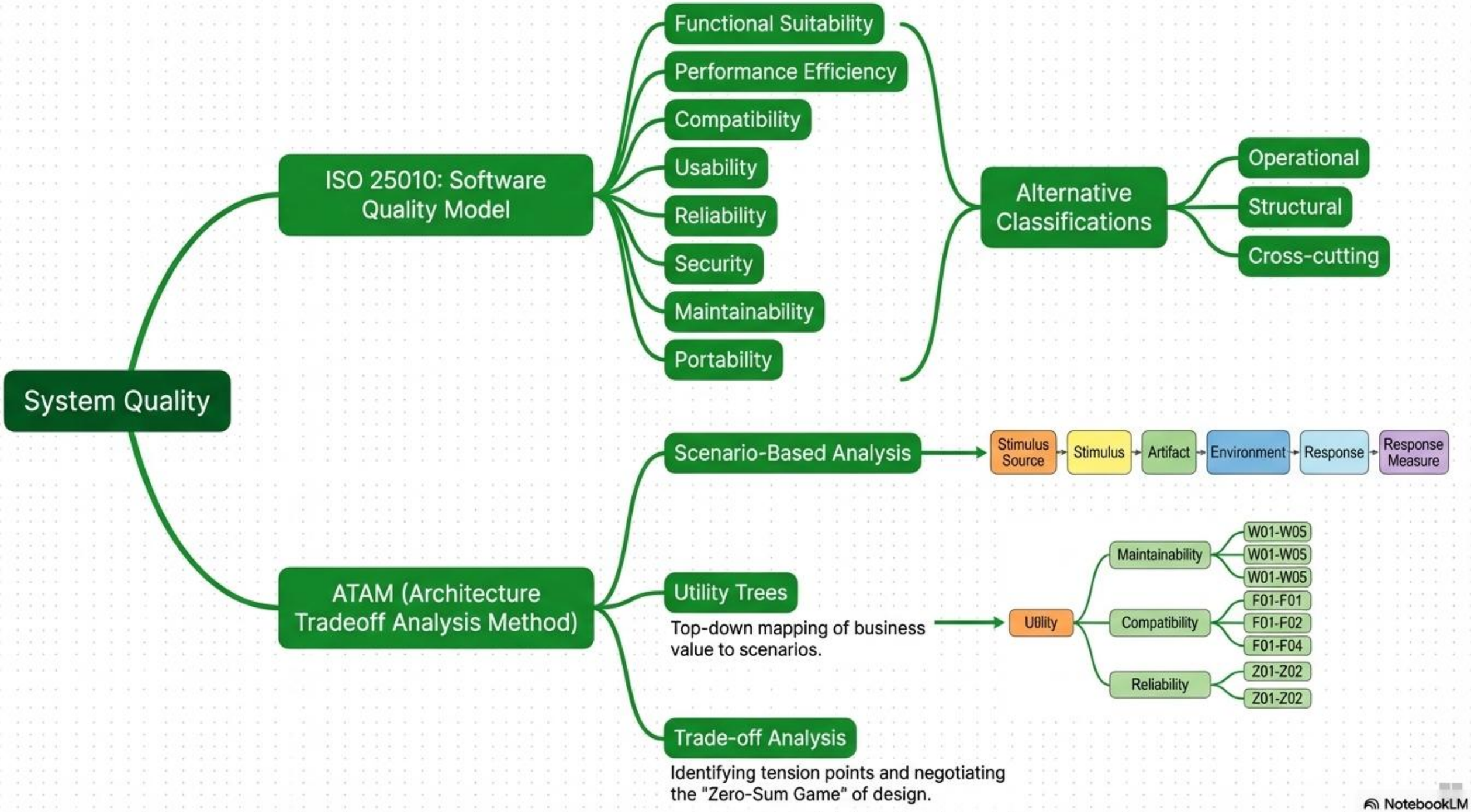
Prioritization & Trade-offs

Utility Trees: Top-down mapping of business value to scenarios.

ATAM: Identifying tension points and negotiating the 'Zero-Sum Game' of design.



NotebookLM


System Quality
ISO 25010: Software Quality Model
Functional Suitability
Performance Efficiency
Compatibility
Usability
Reliability
Security
Maintainability
Portability
Alternative Classifications
Operational
Structural
Cross-cutting
ATAM (Architecture Tradeoff Analysis Method)
Scenario-Based Analysis
Stimulus Source
Stimulus
Artifact
Environment
Response
Response Measure
Utility Trees
Top-down mapping of business value to scenarios.
U0lity
Maintainability
W01-W05
W01-W05
W01-W05
Compatibility
F01-F01
F01-F02
F01-F04
Reliability
Z01-Z02
Z01-Z02
Trade-off Analysis
Identifying tension points and negotiating the "Zero-Sum Game" of design.
NotebookLM

MODULARITY

COHESION

Functional (Best)

Sequential / Communicational

Procedural / Temporal

Logical

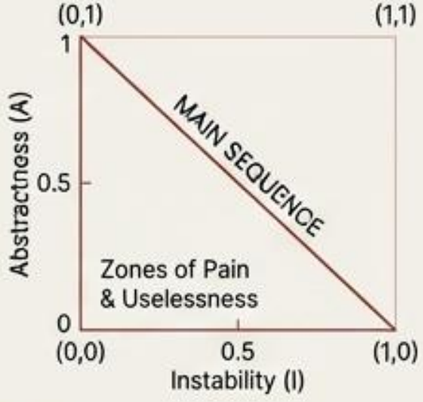
Coincidental (Worst)

COUPLING METRICS

Afferent (Ca) / Efferent (Ce)

Instability: I = Ce / (Ca + Ce)

Abstractness: A = Na / Nc



CONNASCENCE

Static vs. Dynamic

Cost = (Strength × Degree) / Locality

PARTITIONING

- Technical (Layers / Capabilities)
- Domain (Event Storming / Bounded Contexts)

NotebookLM

Modularity Metrics

1 Cohesion (Internal Relatedness)

Best (High Cohesion)

- Functional
- Sequential
- Communicational
- Procedural
- Temporal
- Logical
- Coincidental

Worst (Low Cohesion)

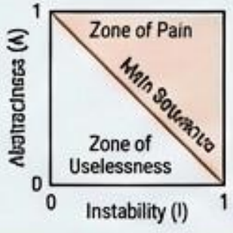
Measure of internal strength of a component's elements.

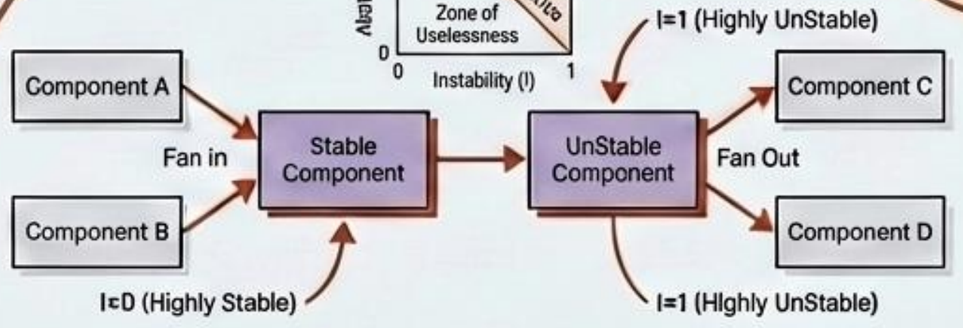
2 Coupling (External Connections)

Afferent (Ca): Who depends on you (incoming).

Instability Formula: I = Ce / (Ca + Ce)

Abstractness Formula: A = Na / (Nc + Na)





System Modularity (The Three C's)

3 Connascence (Change Dependency)

Evaluation Axes: Strength, Degree, and Locality.

Static

- Name
- Type
- Meaning
- Position
- Algorithm

Dynamic

- Execution
- Timing
- Identity

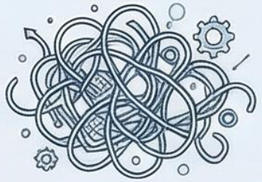
Low (good) → High (bad)

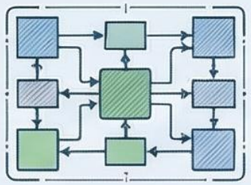
Strength

NotebookLM

The Architect's Toolkit: Measuring Modularity to Tame Complexity

ENTROPY & COMPLEXITY
Natural Tendency





STRUCTURAL SOUNDNESS
Architectural Goal

COHESION (The Internal Bond)

Measure of Internal Relatedness

Cohesion evaluates the extent to which the parts of a single module belong together and contribute to a well-defined task.

Scale

Flow

Functional (Best)
All elements contribute to a single, well-defined task.

Sequential
(Output to Input)

Communicational
(Same Data)

Procedural
(Execution Sequence)

Temporal
(Same Time)

Logical
(Grouped by Logic)

Coincidental (Worst)
Elements are bundled by chance with no meaningful relationship.

COUPLING (External Dependencies)

Measuring Inter-Module Connections

Coupling tracks how modules depend on one another using Afferent (Incoming) and Efferent (outgoing) metrics.

$$\text{Instability (I)} = \frac{C_e}{C_a + C_e}$$

Main Sequence Graph

Abstractness (A)

Afferent Coupling (C_a)
C_a measures who depends on you (incoming), while C_e measures who you depend on (outgoing).

ZONE OF USELESSNESS
No one depends on it, and it performs no concrete action.
High (Unstable), High (Abstract).

ZONE OF PAIN
Hard to change because many depend on it, but it's too right to extend.
Low (Stable), Low (Concrete).

Afferent Coupling (C_a) vs. Efferent Coupling (C_e)
C_a measures who depends on you (incuming), while C_c measures who you depend on (outgoing).

instability (i)

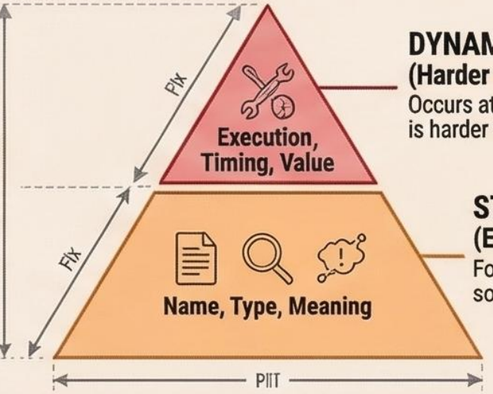
CONNASCENCE (The Ripple Effect)

Correctness through Change

Two components are connascent if a change in one requires the other to be modified to maintain system correctness.

Strength Hierarchy: Static vs. Dynamic

Harder to Fix



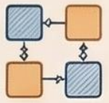
DYNAMIC (Harder to Fix)
Occurs at runtime and is harder to refactor.

STATIC (Easier to Fix)
Found in source code.

The Three Properties

Strength
(ease of refactoring)

Degree
(size of impact/scale)



Locality
(proximity of modules)

Connascence Cost Formula

$$\text{Cost} = \text{Strength} \times \text{Degree} \times \frac{1}{\text{Locality}}$$

The Ultimate Goal: Taming Entropy and Complexity

The primary role of the architect is to use these metrics to combat software entropy, ensuring long-term structural soundness.

The "Least Bad" Architecture

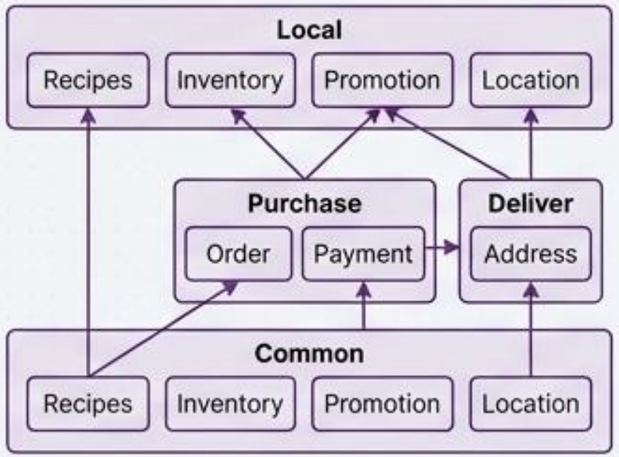
Rather than shooting for a perfect design, architects aim for the "least bad" architecture by balancing trade-offs and minimizing cross-boundary connascence.

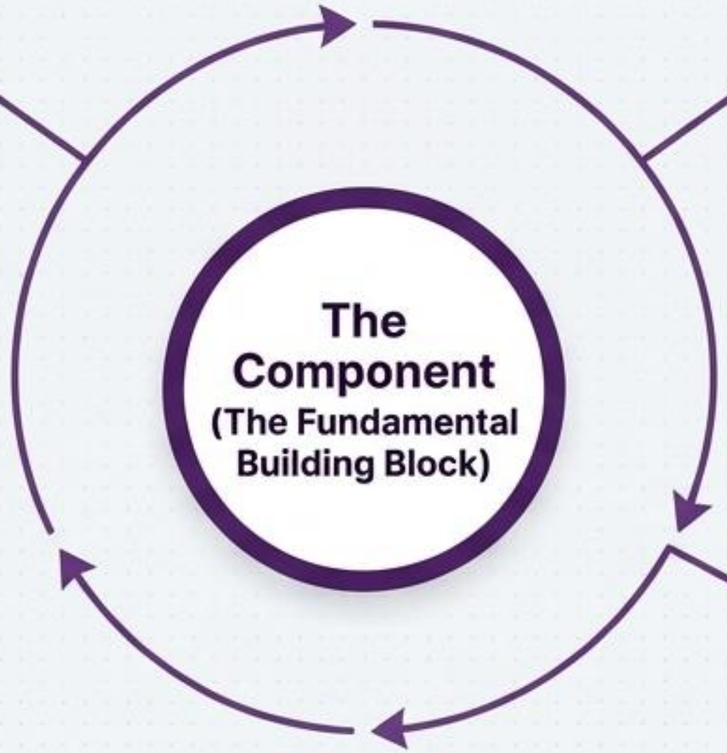
NotebookLM

Component-Based Thinking

Partitioning Styles

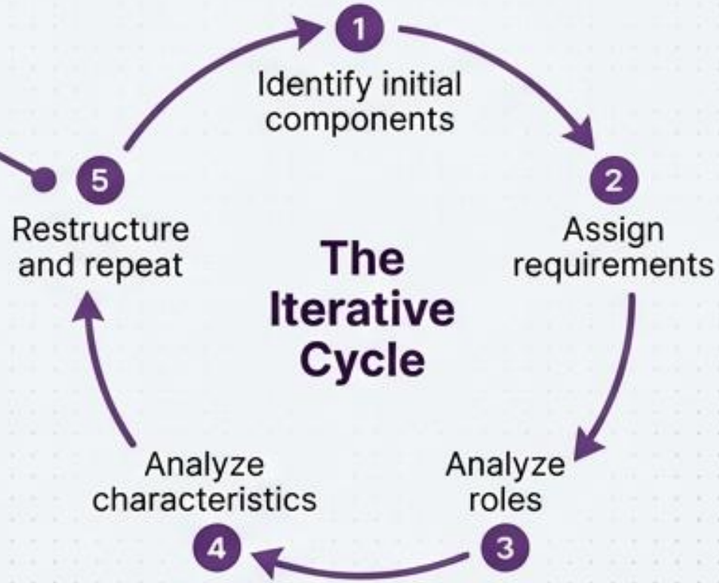
- **Domain Partitioning**: Grouped by business workflows (DDD).
- **Technical Partitioning**: Grouped by technical capabilities (Layered).





The Physical Module

- Standalone, deployable units of related code.
- The lowest level of the system the architect interacts with directly.



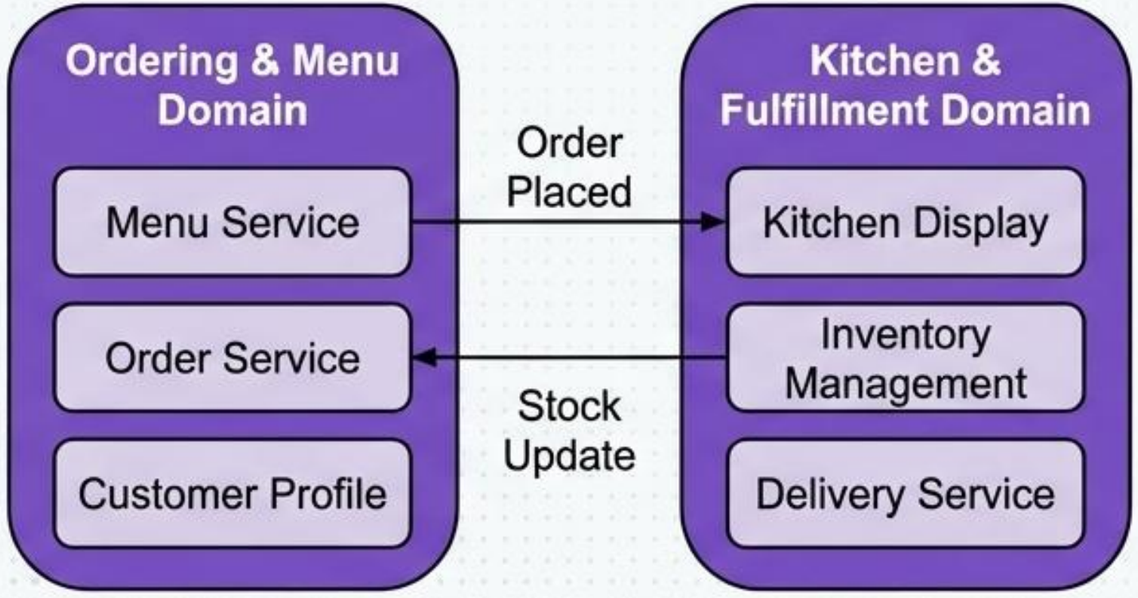
NotebookLM

Component-Based Thinking

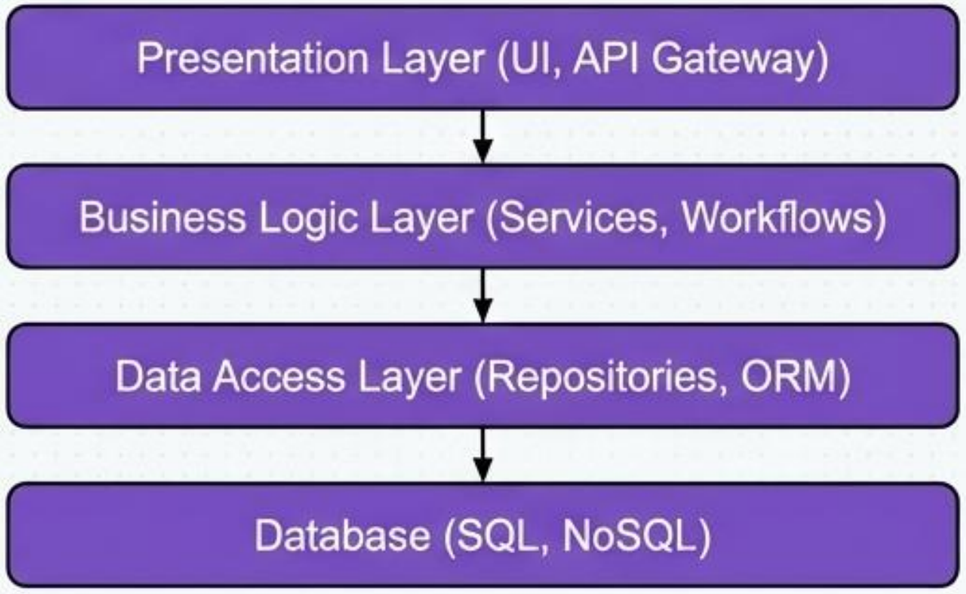
Partitioning Styles

- **Domain Partitioning**: Grouped by business workflows (DDD).
- **Technical Partitioning**: Grouped by technical capabilities (Layered).

Domain Partitioning (e.g., Silicon Sandwiches)

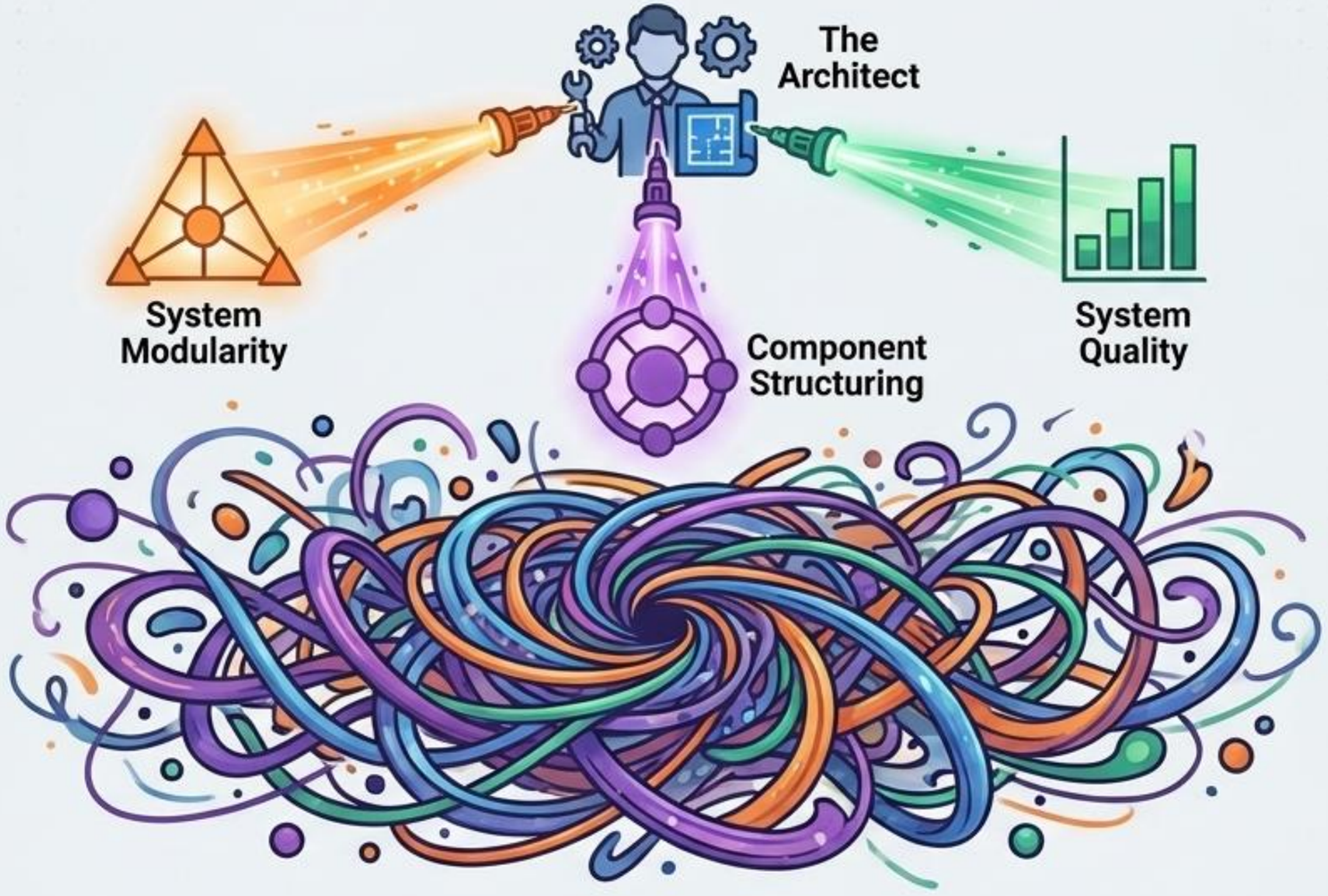


Technical Partitioning (e.g., Layered)



NotebookLM

Taming System Entropy



Core Takeaway: The architect actively employs modularity, component-based thinking, and rigorous quality measurement as essential tools to tame system entropy and maintain order.

NotebookLM